CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati
saradhi@iitg.ac.in

Week #08: STL – Sets, Maps, Unordered Containers

Overview

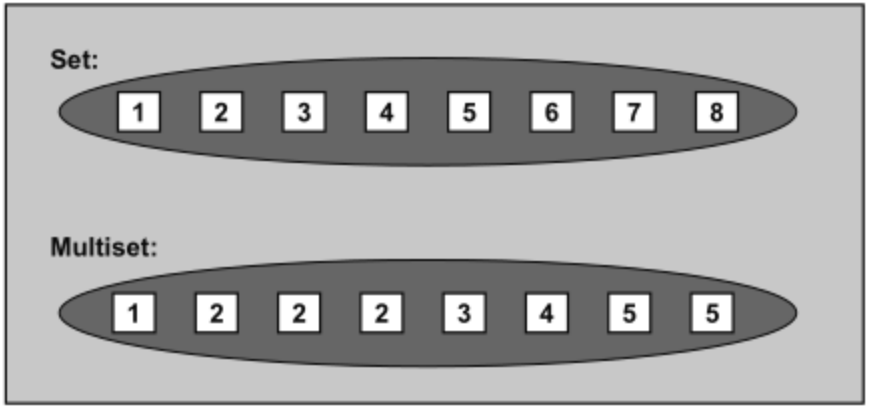
- Standard Template Library
- Sets & Multi-sets
- Maps & Multimaps
- Unordered containers
- Stacks & Queues

Sets & Multisets

Sets & Multisets

- Set and multiset containers sort their elements automatically according to a certain sorting criterion.

- The difference between the two types of containers is that multisets allow duplicates, whereas sets do not



Sets & Multisets

- To use a set or a multiset, you must include the header file

```
#include<set>
```

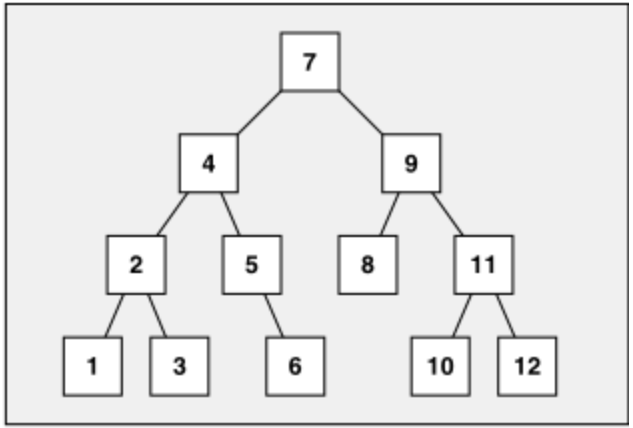
- The elements of a set or a multiset may have any type T that is comparable according to the sorting criterion.
- The optional second template argument defines the sorting criterion
- If a special sorting criterion is not passed, the default criterion less is used.

```
namespace std {
    template <typename T,
              typename Compare = less<T>,
              typename Allocator = allocator<T> >
             class set;

    template <typename T,
              typename Compare = less<T>,
              typename Allocator = allocator<T> >
             class multiset;
}
```

Sets & Multisets - Abilities

- Sets and multisets are usually implemented as balanced binary trees



- The advantage of automatic sorting is that a binary tree performs well when elements with a certain value are searched.
- Search functions have logarithmic time complexity.

Sets & Multisets - Abilities

- Automatic sorting also imposes an important constraint on sets and multisets: You may not change the value of an element directly
- Doing so might compromise the correct order.
- To modify the value of an element, you must remove the element having the old value and insert a new element that has the new value
- The interface reflects this behavior:
  - Sets and multisets don’t provide operations for direct element access
  - Indirect access via iterators has the constraint that, from the iterator’s point of view, the element value is constant.

Sets

Set - Operations

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

Set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s; // empty set
  return 0;
}
```

Set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s = {5, 1, 3, 9, 4};
  return 0;
}
```

Set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  // duplicates removed automatically in sets
  set<int> s = {5, 1, 1, 3, 3, 9, 9, 4, 4};
  return 0;
}
```

Set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s1 = {5, 1, 3};
  // invoke copy constructor
  set<int> s2(s1);
  return 0;
}
```

Set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s1 = {5, 1, 3};
  // moves s1 to s2; s1 is empty
  set<int> s2(move(s1));
  return 0;
}
```

Set - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s1 = {5, 1, 3};

  for( auto it = s1.begin(); it != s1.end(); it++ )
  {
    cout << *it << " ";
  }
  count << endl;
}
```

Set - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s1 = {5, 1, 3};

  if( s1.find(3) != s1.end() )
  {
    cout << "Element 3 is in s1" << endl;
  }
  else
  {
    cout << "Element 3 is NOT in s1" << endl;
  }
}
```

Set - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s1 = {5, 1, 3};

  s1.insert(7);
  s1.insert(8);
  s1.insert(2);
  s1.insert(1); // duplicate. Will not add
}
```

Set - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s1 = {5, 1, 3};

  s1.erase(1);
  s1.erase(s1.begin());

}
```

Set - Iterating

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a set

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<int> s = {20, 10, 40, 30, 50, 10};
  cout << "Elements in the set: ";
  for (auto it = s.begin(); it != s.end(); it++)
  {
    cout << *it << " ";
  }
  cout << std::endl;
  return 0;
}
// Elements in the set: 10 20 30 40 50
```

Set – Specified sorting using a different class

```
#include <iostream>
#include <set>
using namespace std;

struct CompareRatio
{
  bool operator()(const Ratio& r1, const Ratio& r2) const
  {
    // compare ratios
    return (r1.numerator * r2.denominator) < (r2.numerator * r1.denominator);
  }
};

int main()
{
  set<Ratio, CompareRatio> r;
  ratios.insert(Ratio(1, 2)); ratios.insert(Ratio(3, 4));
  ratios.insert(Ratio(2, 3)); ratios.insert(Ratio(1, 3));
  //Iterate & print
}
```

Set – Specified sorting using a function

```
#include <iostream>
#include <set>
using namespace std;

bool compareRatio(const Ratio& r1, const Ratio& r2)
{
  // compare ratios
  return (r1.numerator * r2.denominator) < (r2.numerator * r1.denominator);
}

int main()
{
  set<Ratio, decltype(&compareRatio)> r;
}
```

decltype Extracts the type of the comparison function, making it compatible with std::set.

This is a useful technique when you want to pass function pointers to STL containers.

Set – Specified sorting using lambda function

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  auto ratioCompare = [](const Ratio& r1, const Ratio& r2)
  {
    return r1.compareTo(r2); // a member function in Ratio class
  };
  set<Ratio, decltype(ratioCompare)> r(ratioCompare);
}
```

decltype Extracts the type of the comparison function, making it compatible with std::set.

This is a useful technique when you want to pass function pointers to STL containers.

Set – Specified sorting using member function

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  set<Ratio, decltype(&Ratio::compare)> r(Ratio::compare);
}
```

decltype Extracts the type of the comparison function, making it compatible with std::set.

This is a useful technique when you want to pass function pointers to STL containers.

Multiset

Multiset - Operations

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

Multiset - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms; // empty multiset
  return 0;
}
```

Multiset - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms = {5, 1, , 9, 4};
  return 0;
}
```

Multiset - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  // duplicates handled automatically
  multiset<int> ms = {5, 1, 1, 3, 3, 9, 9, 4, 4};
  return 0;
}
```

Multiset - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms1 = {5, 1, 3};
  // invoke copy constructor
  multiset<int> ms2(ms1);
  return 0;
}
```

Multiset - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms1 = {5, 1, 3};
  // moves ms1 to ms2; ms1 is empty
  multiset<int> ms2(move(ms1));
  return 0;
}
```

Multiset - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms1 = {5, 1, 3};

  for( auto it = ms1.begin(); it != s1.end(); it++ )
  {
    cout << *it << " ";
  }
  count << endl;
}
```

Multiset - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms1 = {5, 1, 3};

  if( ms1.find(3) != ms1.end() )
  {
    cout << "Element 3 is in ms1" << endl;
  }
  else
  {
    cout << "Element 3 is NOT in ms1" << endl;
  }
}
```

Multiset - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms1 = {5, 1, 3};

  ms1.insert(7);
  ms1.insert(7);
  ms1.insert(8);
  ms1.insert(8);
  ms1.insert(2);
  ms1.insert(2);
  ms1.insert(1);

}
```

Multiset - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> s1 = {5, 1, 1, 1, 1, 1, 3};

  s1.erase(1); // removes all occurrences of 1
  s1.erase(s1.begin()); // removes 1st element
}
```

Multiset - Iterating

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multiset

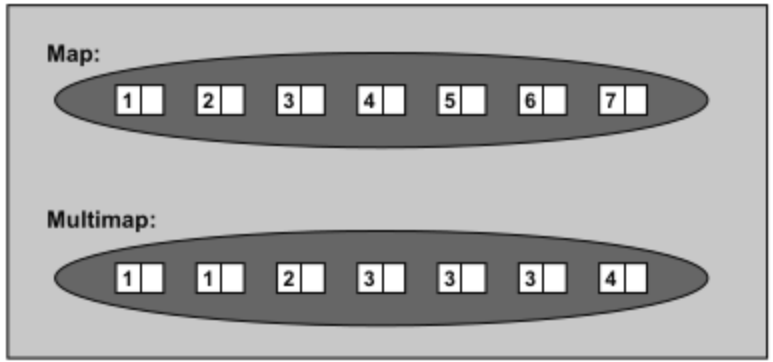
```
#include <iostream>
#include <set>
using namespace std;

int main()
{
  multiset<int> ms = {20, 20, 10, 40, 30, 50, 10};
  cout << "Elements in the multiset: ";
  for (auto it = ms.begin(); it != ms.end(); it++)
  {
    cout << *it << " ";
  }
  cout << std::endl;
  return 0;
}
// Elements in the multiset: 10 10 20 20 30 40 50
```

Maps & Multimaps

Maps & Multimaps

- Maps and multimaps are containers that manage key/value pairs as elements.
- These containers sort their elements automatically, according to a certain sorting criterion that is used for the key.



Maps & Multimaps

- To use a map or multimap, you must include the header file

```
#include<map>
```

- The difference between the two is that multimaps allow duplicates, whereas maps do not

```
namespace std {
    template <typename Key, typename T,
              typename Compare = less<Key>,
              typename Allocator = allocator<pair<const Key,T> > >
             class map;

    template <typename Key, typename T,
              typename Compare = less<Key>,
              typename Allocator = allocator<pair<const Key,T> > >
             class multimap;
}
```

Maps & Multimaps

- The first template parameter is the type of the element's key
- The second template parameter is the type of the element's associated value.
- The elements of a map or a multimap may have any types Key and T that meet the following two requirements:
  - Both key and value must be copyable or movable
  - The key must be comparable with the sorting criterion
  - The optional third template parameter defines the sorting criterion

Maps

Maps - Operations

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

Maps - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m; // empty map
  return 0;
}
```

Map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m = {
                        {2, "two"},
                        {0, "zero"},
                        {4, "four"}
                       };
  return 0;
}
```

Map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  // duplicates removed automatically in sets
 map<int, string> m = {
                          {2, "two"},
                          {0, "zero"},
                          {4, "four"},
                          {2, "two"},
                          {0, "zero"},
                          {4, "four"}
                         };
  return 0;
}
```

Map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = {
                         {2, "two"},
                         {0, "zero"},
                         {4, "four"}
                        };
  map<int, string> m2(m1);
  return 0;
}
```

Map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = {
                         {2, "two"},
                         {0, "zero"},
                         {4, "four"}
                        };
  map<int, string> m2(move(m1));
  return 0;
}
```

Map - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = { {1, "one"}, {5, "five"}};

  cout << m1[5] << endl;
  cout << m1[1] << endl;
}
```

Map - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = { {1, "one"}, {5, "five"}};

  cout << m1.at(5) << endl;
  cout << m1.at(1) << endl;
}
```

Map - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = { {1, "one"}, {5, "five"}};

  for( const auto& pair : m )
  {
    cout << pair.first << ":" << pair.second<< endl;
  }
}
```

Map - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = { {1, "one"} };
  m1.insert({5, "five"});
  m1.insert({6, "six"});
  m1.insert({7, "seven"});
}
```

Map - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m1 = { {1, "one"} };
  m1[5] = "five";
  m1[6] = "six";
  m1[7] = "seven";
}
```

Map - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m = { {5, "five"} };

  m1.erase(5);

}
```

Map - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  map<int, string> m = { {5, "five"} };
  auto it = m.find(1);
  if (it != m.end() )
  {
    m.erase(it);
  }
}
```

Set - Iterating

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a map

```cpp
#include<iostream>
#include<map>
using namespace std;
int main(int argc, char *argv[])
{
    map<int, string> m = {{1, "one"}, {2, "two"}, {3, "three"}};
    cout << "Value for key 1: " << m[1] << endl;
    cout << "Value for key 2: " << m.at(2) << endl;
    m.insert({4, "four"});
    m[5] = "five";
    for (const auto& pair : m)
    {
        cout << pair.first << " : " << pair.second << endl;
    }
    m.erase(3);
    m.erase(m.find(4));
    for (const auto& pair : m)
    {
        cout << pair.first << " : " << pair.second << endl;
    }
    return 0;
}
```

Multimap

Multimap - Operations

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

Multimap - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> mm; // empty multimap
  return 0;
}
```

Multimap - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> mm = { {5, "five"} };
  return 0;
}
```

Multimap - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  // duplicates handled automatically
  multimap<int, string> mm = {
                        {1, "one"},
                        {1, "one"},
                        {2, "two"},
                        {2, "two"},
                        {2, "two}
                      };
  return 0;
}
```

Multimap - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> mm1 = { {5, "five"} };
  // invoke copy constructor
  multimap<int, string> mm2(mm1);
  return 0;
}
```

Multimap - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> mm1 = { {5, "five"} };
  // moves mm1 to mm2; mm1 is empty
  multimap<int, string> mm2(move(mm1));
  return 0;
}
```

Multimap - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string>  m1 = { {5, "five"},
                               {6, "six"},
                               {7, "seven"}
                             };
  for( const auto &pair : m1 )
  {
    cout << pair.first << " : " << pair.second;
  }
  cout << endl;
}
```

Multimap - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> mm1 = { {5, "five"} };

  auto it = mm1.find(5);
  cout << it->second << endl;
}
```

Multimap - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> ms1 = { {5, "five"} };

  ms1.insert({7, "seven"});
  ms1.insert({7, "seven"});
  ms1.insert({8, "eight"});
  ms1.insert({8, "eight"});
  ms1.insert({2, "two"});
  ms1.insert({2, "two"});
  ms1.insert({1, "one"});
}
```

Multimap - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> s1 = { {5, "five"},
                               {3, "three"}
                             };

  s1.erase(5); // removes all occurrences of 5
  s1.erase(s1.begin()); // removes 3
}
```

Multimap - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string> s1 = { {5, "five"},
                               {3, "three"}
                             };
 auto it = s1.find(1);
 if (it != s1.end())
 {
   s1.erase(it);
 }
}
```

Multimap - Iterating

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over a multimap

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
  multimap<int, string>  m1 = { {5, "five"},
                               {6, "six"},
                               {7, "seven"}
                             };
  for( const auto &pair : m1 )
  {
    cout << pair.first << " : " << pair.second;
  }
  cout << endl;
}
```

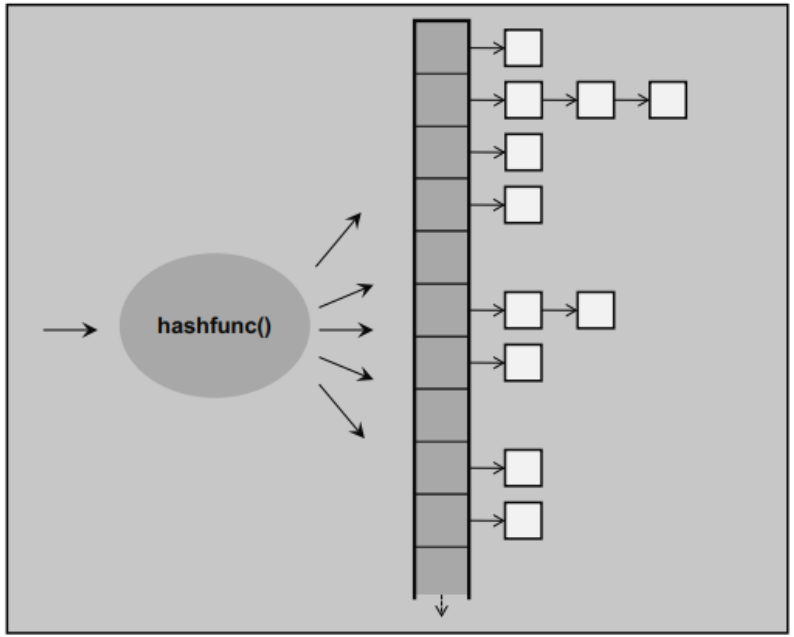
Unordered Containers

Unordered Containers

- **unordered containers** are containers that do not maintain any specific order of elements but instead provide fast access through **hashing**.
- These include
  - `unordered_set`
  - `unordered_multiset`
  - `unordered_map`
  - `unordered_multimap`
- `I will describe unordered_set and unordered_map in this tutorial`

unordered_set

Unordered Containers




```
You must use the header file:

#include<unordered_set>
```

Unordered Containers

- **unordered containers** are containers that do not maintain any specific order of elements but instead provide fast access through **hashing**.
- These include
  - `unordered_set`
  - `unordered_multiset`
  - `unordered_map`
  - `unordered_multimap`
- `I will describe unordered_set and unordered_map in this tutorial`

unordered_set - Operations

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

unordered_set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> us; // empty set
  return 0;
}
```

unordered_set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  Unordered_set<int> us = {5, 1, 3, 9, 4};
  return 0;
}
```

unordered_set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  // duplicates removed automatically in sets
  unordered_set<int> us = {5, 1, 1, 3, 3, 9, 4, 4};

  return 0;
}
```

unordered_set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> us1 = {5, 1, 3};
  // invoke copy constructor
  unordered_set<int> us2(s1);
  return 0;
}
```

unordered_set - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> us1 = {5, 1, 3};
  // moves us1 to us2; us1 is empty
  unordered_set<int> us2(move(us1));
  return 0;
}
```

unordered_set - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> s1 = {5, 1, 3};

  for (const auto& i : s1)
  {
    cout << i << " ";
  }
  count << endl;
}
```

unordered_set - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> s1 = {5, 1, 3};

  if( s1.find(3) != s1.end() )
  {
    cout << "Element 3 is in s1" << endl;
  }
  else
  {
    cout << "Element 3 is NOT in s1" << endl;
  }
}
```

unordered_set - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> s1 = {5, 1, 3};

  s1.insert(7);
  s1.insert(8);
  s1.insert(2);
  s1.insert(1); // duplicate. Will not add
}
```

unordered_set - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> s1 = {5, 1, 3};

  s1.erase(1);
  s1.erase(s1.begin());

}
```

unordered_set - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> s1 = {5, 1, 3};

  auto it = s1.find(5);
  if (it != s1.end())
  {
    s1.erase(it);
  }
}
```

unordered_set - Iterating

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over unordered_set

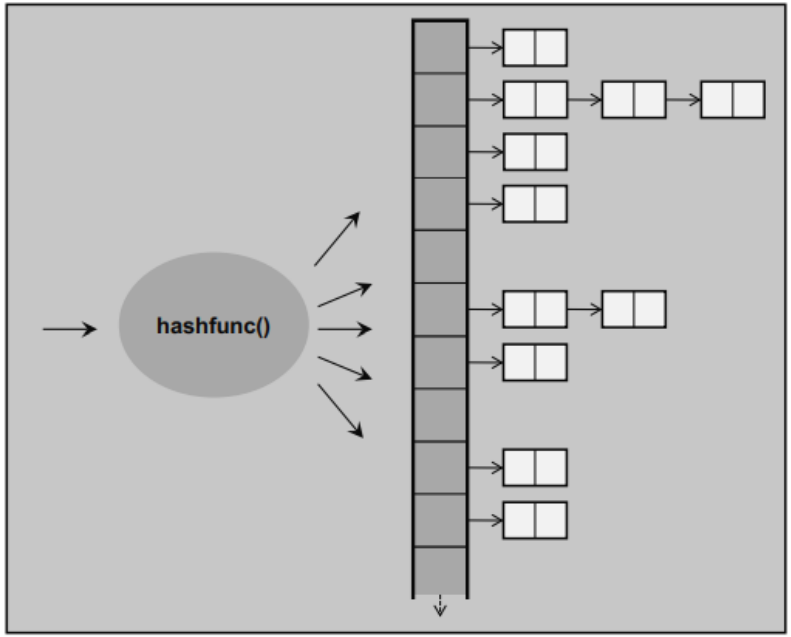
```
#include <iostream>
#include <unordered_set>
using namespace std;

int main()
{
  unordered_set<int> s = {20, 10, 40, 30, 50, 10};
  cout << "Elements in the set: ";
  for (const auto& it : s)
  {
    cout << it << " ";
  }
  cout << std::endl;
  return 0;
}
// Elements in the set: 50 30 40 10 20
```

unordered_map

Unordered Maps

- **unordered_map** is an unordered associative container that contains key-value pairs. It uses a **hash table** to provide fast access based on the keys.



unordered_map

- To use an unordered_map, unordered_multimap you must include the header file

```
#include<unordered_map>
```

- The difference between the two is that unordered_multimap allow duplicates, whereas unordered_map do not

unordered_map

unordered_map - Operations

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

unordered_map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m; // empty map
  return 0;
}
```

unordered_map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m = {
                              {2, "two"},
                              {0, "zero"},
                              {4, "four"}
                             };
  return 0;
}
```

unordered_map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  // duplicates removed automatically in sets
 unordered_map<int, string> m = {
                            {2, "two"},
                            {0, "zero"},
                            {4, "four"},
                            {2, "two"},
                            {0, "zero"},
                            {4, "four"}
                           };
  return 0;
}
```

unordered_map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = {
                            {2, "two"},
                            {0, "zero"},
                            {4, "four"}
                           };
  map<int, string> m2(m1);
  return 0;
}
```

unordered_map - Declaration

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = {
                            {2, "two"},
                            {0, "zero"},
                            {4, "four"}
                           };
  unordered_map<int, string> m2(move(m1));
  return 0;
}
```

unordered_map - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = { {1, "one"}, {5, "five"}};

  cout << m1[5] << endl;
  cout << m1[1] << endl;
}
```

unordered_map - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = { {1, "one"}, {5, "five"}};

  cout << m1.at(5) << endl;
  cout << m1.at(1) << endl;
}
```

unordered_map - Accessing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = { {1, "one"}, {5, "five"}};

  for( const auto& pair : m )
  {
    cout << pair.first << ":" << pair.second<< endl;
  }
}
```

unordered_map - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = { {1, "one"} };
  m1.insert({5, "five"});
  m1.insert({6, "six"});
  m1.insert({7, "seven"});
}
```

unordered_map - Adding

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m1 = { {1, "one"} };
  m1[5] = "five";
  m1[6] = "six";
  m1[7] = "seven";
}
```

Unordered_map - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m = { {5, "five"} };

  m1.erase(5);

}
```

Unordered_map - Removing

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main()
{
  unordered_map<int, string> m = { {5, "five"} };
  auto it = m.find(1);
  if (it != m.end() )
  {
    m.erase(it);
  }
}
```

unordered_map - Iterating

- Declaration
- Accessing elements
- Adding elements
- Removing elements
- Iterating over an unordered_map

```cpp
#include <iostream>
#include <unordered_map>
using namespace std;

int main(int argc, char *argv[])
{
    unordered_map<int, string> umap = {{1, "one"}, {2, "two"}, {3, "three"}};

    cout << "Value of key=1: " << umap[1] << endl;

    umap[4] = "four";
    umap.insert({5, "five"});

    cout << "Unordered map elements:" << endl;
    for (const auto& pair : umap)
    {
        cout << pair.first << " : " << pair.second << endl;
    }

    umap.erase(2);

    cout << "Map after removal:" << endl;
    for (const auto& pair : umap) {
        cout << pair.first << " : " << pair.second << endl;
    }

    return 0;
}
```

Stack Implementation Using `vector`

Stack – Implementation using vector

- This example demonstrate the implementation of stack using vector
- push_back - Appends an element at the end
- pop_back - Removes the last element

```
template <typename T>
class Stack
{
private:
    vector<T> data;
public:
    void push(const T& value)
    {
        data.push_back(value);
    }
    void pop()
    {
      if (!empty())
        data.pop_back();
      else
        cerr << "Stack is empty." << endl;
    }
};
```

Stack – Implementation using vector

- back: returns a reference to the last element in the vector.

```
template <typename T>
class Stack
{
private:
    vector<T> data;
public:
    T& top()
    {
      if (!empty())
        return data.back();
      else
        throw out_of_range("Stack is empty");
    }
};
```

Stack – Implementation using vector

- empty: Returns whether the container is empty
- size: Returns the current number of elements

```
template <typename T>
class Stack
{
private:
    vector<T> data;
public:
    bool empty() const
    {
      return data.empty();
    }
    size_t size() const
    {
      return data.size();
    }
};
```

Questions?

Thank You